

Arquitectura de Datos - AutoVentas Pro

Patron Repository (Arquitectura Hexagonal)

Separacion estricta entre **contrato** (interfaz en dominio) e **implementacion** (infraestructura).

```
Dominio
└─ repositories/      ← Interfaces
   └─ ICotizacionRepository.ts

Infraestructura
└─ repositories/      ← Implementaciones concretas
   └─ cotizacion.supabase.repository.ts
```

Por que?

- El dominio no depende de Prisma ni de ninguna tecnologia externa
- Si se cambia Prisma por otro ORM, solo se cambia la carpeta `infrastructure/repositories/`
- Los tests pueden mockear las interfaces sin tocar la DB

1. Interfaces (Dominio)

Definen metodos minimos CRUD + busquedas especificas.

```
// src/domain/repositories/ICotizacionRepository.ts
export interface CotizacionRepository {
  save(cotizacion: Cotizacion): Promise<void>;
  update(cotizacion: Cotizacion): Promise<void>;
  delete(id: string): Promise<boolean>;
  findById(id: string): Promise<Cotizacion | null>;
  findAll(): Promise<Cotizacion[]>;
  findActivas(): Promise<Cotizacion[]>;
  findByVehiculo(vehiculoId: string): Promise<Cotizacion[]>;
  count(): Promise<number>;
}
```

2. Conexion a Supabase (Singleton)

```
// src/infrastructure/database/prisma.service.ts
import pg from 'pg';
import { PrismaPg } from '@prisma/adapters-pg';
import { PrismaClient } from '../../../generated/prisma/client.js';

let prisma: PrismaClient;
```

```
export function getPrisma(): PrismaClient {
  if (!prisma) {
    const pool = new pg.Pool({
      host: process.env['DB_HOST'],
      port: Number(process.env['DB_PORT']),
      database: process.env['DB_NAME'],
      user: process.env['DB_USER'],
      password: process.env['DB_PASSWORD'],
    });
    const adapter = new PrismaPg(pool);
    prisma = new PrismaClient({ adapter });
  }
  return prisma;
}
```

- Usa **pg.Pool** (pool de conexiones) en vez de connection string directa
- **PrismaPg** es el adapter de Prisma 7 para PostgreSQL
- **Singleton** — una sola instancia de PrismaClient en toda la app

3. Repositorio concreto (Implementacion)

Cada repositorio sigue este patron:

```
// src/infrastructure/repositories/cotizacion.supabase.repository.ts
import { Cotizacion } from '../../../domain/entities/Cotizacion.js';
import { getPrisma } from '../database/prisma.service.js';

export class CotizacionSupabaseRepository implements CotizacionRepository {
  async save(cotizacion: Cotizacion): Promise<void> {
    const prisma = getPrisma();
    await prisma.cotizacion.create({ data: cotizacion.toJSON() });
    //                               ↑ modelo Prisma   ↑ entidad → JSON
  }

  async findById(id: string): Promise<Cotizacion | null> {
    const prisma = getPrisma();
    const data = await prisma.cotizacion.findUnique({ where: { id } });
    return data ? Cotizacion.desdeDatos(data) : null;
    //                               ↑ JSON → entidad de dominio
  }
}
```

Reglas del patron:

1. **getPrisma()** para obtener instancia de Prisma
2. **entity.toJSON()** para convertir entidad → datos planos hacia Prisma
3. **Entity.desdeDatos(dbRow)** para convertir fila de DB → entidad de dominio
4. **Nunca** se usan tipos de Prisma fuera de esta carpeta

4. Wire de dependencias (Routes)

Las rutas importan repositorios, crean servicios y controladores:

```
// src/interfaces/routes/cotizacionRoutes.ts
const cotizacionRepo = new CotizacionSupabaseRepository();
const vehiculoRepo = new VehiculoSupabaseRepository();
const planRepo = new PlanFinanciamientoSupabaseRepository();
const service = new CotizacionService(cotizacionRepo, vehiculoRepo, planRepo);
const controller = new CotizacionController(service);

router.get('/', controller.listar);
router.post('/', controller.crear);
router.post('/simular', controller.simular);
```

Los repositorios **nunca** se instancian dentro de servicios — se inyectan desde afuera (dependency injection manual).

5. Supabase Storage

Para imagenes de vehiculos, no para datos estructurados.

```
// src/infrastructure/storage/supabase.storage.ts
import { createClient } from '@supabase/supabase-js';

const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY);

export async function uploadFile(buffer: Buffer, fileName: string, mimeType: string): Promise<string> {
  // 1. Sube archivo al bucket "vehiculos"
  await supabase.storage.from('vehiculos').upload(filePath, buffer, {
    contentType: mimeType });

  // 2. Retorna URL publica
  const { data } = supabase.storage.from('vehiculos').getPublicUrl(filePath);
  return data.publicUrl;
}
```

- Bucket **publico** (RLS permite INSERT/SELECT/DELETE para **anon**)
- Se accede via **SUPABASE_ANON_KEY**, no via **service_role**
- La URL publica se guarda en el campo **imagen** del Vehiculo

6. Flujo completo (ejemplo: crear cotizacion)

```
POST /api/cotizaciones
→ CotizacionController.crear()
→ CotizacionService.crearCotizacion()
```

→ VehiculoRepo.findById() ← busca vehiculo

→ PlanRepo.findById() ← busca plan

→ Plan.calcularCuota() ← logica dominio

→ TablaAmortizacion.generar() ← logica dominio

→ ClienteInfo() constructor ← value object

→ Cotizacion.crear() ← entidad

→ CotizacionRepo.save() ← persiste en DB

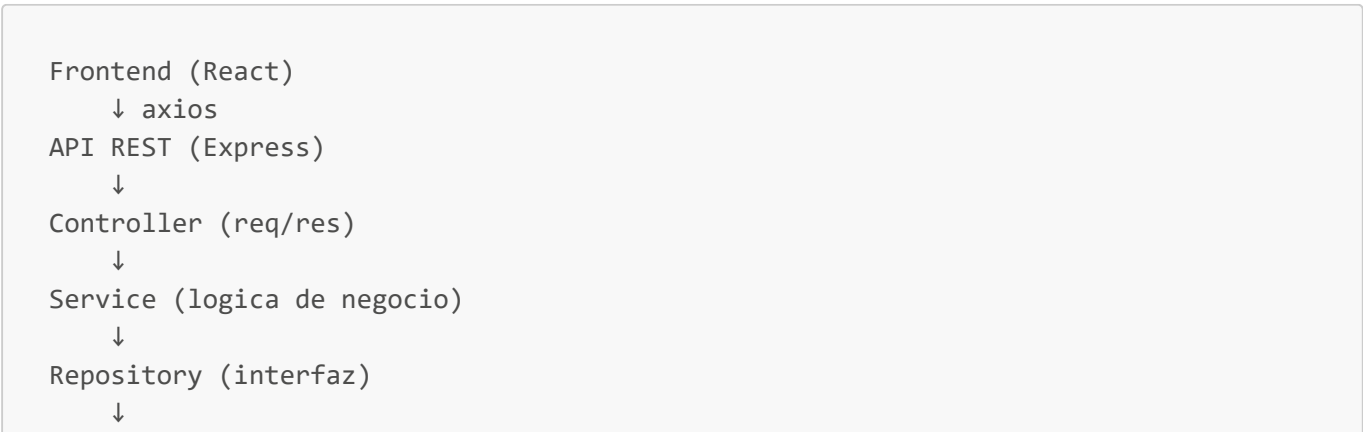
→ getPrisma()

→ prisma.cotizacion.create()

7. Repositorios existentes

Interface	Implementacion	Metodos principales
empleado.repository.ts	empleado.supabase.repository.ts	CRUD + findAll, count
usuario.repository.ts	usuario.supabase.repository.ts	CRUD + findByUsuario, findByRol, count
IVehiculoRepository.ts	vehiculo.supabase.repository.ts	CRUD + findByTipo, findDisponibles, findActivos, count
IPlanFinanciamientoRepository.ts	plan.supabase.repository.ts	CRUD + findActivos, count
ICotizacionRepository.ts	cotizacion.supabase.repository.ts	CRUD + findActivas, findByVehiculo, count
IVentaRepository.ts	venta.supabase.repository.ts	CRUD + findByAsesor, findVentasPorPeriodo, findTopVehiculos, count
IAsesorRepository.ts	AsesorRepositoryImpl.ts	No usado (modulo muerto)

8. Diagrama de flujo de datos



```
graph TD; A[Infrastructure Repository (Prisma)] --> B[PrismaPg Adapter]; B --> C[pg.Pool]; C --> D[Supabase PostgreSQL (AWS us-east-2)];
```

Infrastructure Repository (Prisma)

↓

PrismaPg Adapter

↓

pg.Pool

↓

Supabase PostgreSQL (AWS us-east-2)